

4. Configuration JPA (unité de persistance)

Étape 2 — EILMS : Database Persistence Layer (JDBC, PostgreSQL, JPA/Hibernate)

Fichier concerné : src/main/resources/META-INF/persistence.xml

C'est le descripteur de déploiement standard JPA. Il définit l'unité de persistance, les propriétés de connexion à la base, et les instructions de dialecte ORM.

```
<persistence xmlns="https://jakarta.ee/xml/ns/persistence" version="3.0">
  <persistence-unit name="eilms-pu">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <class>com.eilms.entity.WarehouseItemEntity</class>

    <properties>
      <!-- Driver et identifiants de connexion -->
      <property name="jakarta.persistence.jdbc.driver" value="org.postgresql.Driver"/>
      <property name="jakarta.persistence.jdbc.url"
        value="jdbc:postgresql://localhost:5432/eilms_db"/>
      <property name="jakarta.persistence.jdbc.user" value="postgres"/>
      <property name="jakarta.persistence.jdbc.password" value="postgres_password"/>

      <!-- Dialecte et génération automatique du schéma -->
      <property name="hibernate.dialect"
        value="org.hibernate.dialect.PostgreSQLDialect"/>
      <property name="hibernate.hbm2ddl.auto" value="update"/>
      <property name="hibernate.show_sql" value="true"/>
      <property name="hibernate.format_sql" value="true"/>
    </properties>
  </persistence-unit>
</persistence>
```

Explication détaillée

persistence-unit name="eilms-pu" : nomme l'unité de persistance — c'est ce nom (« eilms-pu ») qui sera utilisé dans le code Java pour récupérer la configuration (voir `Persistence.createEntityManagerFactory("eilms-pu")`).

provider : indique quelle implémentation JPA utiliser concrètement — ici Hibernate.

class : liste les entités gérées par cette unité de persistance (ici, `WarehouseItemEntity`).

jakarta.persistence.jdbc.* : les mêmes informations de connexion que dans `DatabaseConfig.java` (driver, url, user, password), mais cette fois lues par Hibernate lui-même plutôt que gérées manuellement.

hibernate.dialect = PostgreSQLDialect : indique à Hibernate de générer du SQL spécifique à PostgreSQL (types de colonnes, séquences, verrouillage transactionnel...). Exemple : le dialecte change la façon dont un auto-incrément est généré selon qu'on utilise PostgreSQL, MySQL ou Oracle.

hibernate.hbm2ddl.auto = update : à chaque démarrage de l'application, Hibernate compare les entités annotées avec le schéma PostgreSQL existant, et crée/modifie automatiquement les tables si nécessaire. Pratique en développement, mais risqué en production (préférer des migrations contrôlées comme Flyway/Liquibase).

hibernate.show_sql / format_sql : affichent dans la console le SQL généré par Hibernate, formaté proprement — très utile pour déboguer et comprendre ce qui se passe réellement derrière l'ORM.

Analyse pédagogique du professeur

Dialecte PostgreSQL : informe Hibernate de générer une syntaxe SQL spécifique à PostgreSQL (types de colonnes, générateurs de séquences, mécanismes de verrouillage transactionnel).

Génération automatique du schéma (hbm2ddl.auto) : mettre cette valeur à « update » demande à Hibernate de comparer les définitions d'entités annotées avec les métadonnées PostgreSQL existantes, et de générer/modifier automatiquement le schéma DDL au démarrage de l'application.